BÁO CÁO THỰC HÀNH KỸ THUẬT PHẦN MỀM

CHALLENGE A: PERSISTENT MEMORY TRONG AGENT HARNESS

THÔNG TIN CHUNG

- **Môn học:** SE373 – Kỹ thuật xây dựng Agentic AI
- **Nhóm thực hiện:** Nhóm 20

1. GIỚI THIỆU VÀ PHẠM VI

1.1. Bản chất vấn đề: LLM Stateless và Agent Stateful

Trong kiến trúc ứng dụng dựa trên mô hình ngôn ngữ lớn (LLM), bản thân mô hình nền tảng là một hàm phi trạng thái (stateless function). Mỗi khi gửi một yêu cầu suy luận qua API, mô hình chỉ tiếp nhận và xử lý dữ liệu được cung cấp trong chính lượt gọi (request payload) hiện tại; nó không thể tự lưu trữ hay nhớ lại những tương tác, trao đổi hoặc quyết định ở các phiên làm việc trước đó.

Để xây dựng một tác tử phần mềm có trạng thái (stateful agent), lớp điều phối bao quanh mô hình (thường gọi là **Agent Harness**) phải đảm nhiệm toàn bộ vòng đời quản lý thông tin:
1. **Lưu trữ (Persistence):** Quyết định thông tin nào có giá trị lâu dài cần được ghi vào bộ nhớ cố định trên đĩa.
2. **Truy xuất (Retrieval):** Lựa chọn và lọc ra những thông tin thực sự liên quan đến tác vụ và phạm vi dự án hiện tại.
3. **Nạp ngữ cảnh (Injection):** Định dạng và đưa thông tin đã truy xuất vào khối ngữ cảnh (`context`) gửi kèm trong request kế tiếp cho LLM.

1.2. Mục tiêu Challenge A và phạm vi thực hành

Theo yêu cầu đặc tả đề bài, mục tiêu cốt lõi của Challenge A là xây dựng cơ chế để agent:
- Ghi nhớ được các quy ước kỹ thuật của dự án qua nhiều phiên làm việc độc lập (sessions).
- Cập nhật nhất quán khi quy ước có sự thay đổi.

Đề bài yêu cầu chứng minh hai trường hợp bắt buộc:
- **Chứng minh 1:** Session mới vẫn sử dụng đúng các quy ước kỹ thuật đã thống nhất trước đó.
- **Chứng minh 2:** Khi một quy ước được thay đổi, các session sau đó phải sử dụng quy ước mới.

Phạm vi triển khai của đồ án:
- **Lớp lõi Persistence & CLI:** Module memory_agent.py và script kiểm chứng tự động demo.py, tập trung chứng minh hai yêu cầu trên qua 5 tiến trình hệ điều hành (OS subprocess) độc lập.
- **Lớp ứng dụng mở rộng (Web Chatbot):** Module chat_engine.py, webapp.py và giao diện web/ cung cấp giao diện hội thoại đa phiên chat, nhận diện quyết định tự nhiên, trực quan hóa injection và xuất bằng chứng JSON.
- **Giới hạn phạm vi:** Không triển khai các bài tập nâng cao khác như Challenge B (Compaction / Token Savings) hay Challenge C (Model Router). Báo cáo phân biệt rõ ràng giữa kết quả kiểm chứng kiến trúc harness/offline đã chạy thành công với việc kiểm nghiệm trên mô hình LLM thương mại thật.

2. PHÂN TÍCH ĐỀ BÀI VÀ TIÊU CHÍ ĐÁNH GIÁ

2.1. Phân loại các nhóm yêu cầu kỹ thuật

Việc đánh giá cần bám sát cấu trúc yêu cầu để phân biệt rõ giữa điều kiện bắt buộc, định hướng kỹ thuật và phần tính năng mở rộng:

1. **Yêu cầu bắt buộc cốt lõi:**
   - Đạt hai chứng minh: (1) Session mới giữ đúng quy ước cũ; (2) Khi quy ước đổi, session sau áp dụng quy ước mới.
   - Cho phép tái sử dụng tác tử đã có.

- Mốc thời lượng 30 phút được xem như định hướng thực hành; báo cáo không tự tuyên bố thời gian làm đúng 30 phút nếu không có số liệu đo lường thực tế.

2. **Định hướng thiết kế và kiểm chứng:**
   - *Lưu ở đâu:* Các phương án lưu trữ khả thi gồm file Markdown, JSON hoặc SQLite. Bài thực hành chọn SQLite vì khả năng truy vấn schema rõ ràng, hỗ trợ transaction nguyên tử và ràng buộc chỉ mục độc nhất.
   - *Ai ghi:* Agent tự gọi công cụ (tool call) hoặc harness tự trích xuất từ cuộc trò chuyện.
   - *Khi nào nạp:* Nạp lúc mở session hoặc truy xuất động (retrieve) khi cần.
   - *Xử lý thay đổi:* Xét trường hợp điển hình khi yêu cầu thay đổi: Session 1 chốt `Authentication: JWT`, Session 2 đổi sang `session cookie`. Giải pháp hợp lý và chuẩn mực là: **Đánh dấu JWT đã bị thay thế (superseded), lưu nguồn gốc (source) và thời điểm thay thế**, không giữ cả hai quy ước mâu thuẫn cùng lúc và không xóa sạch dấu vết lịch sử.
   - *Kiểm chứng thực tế:* In ra phần memory đã được inject vào context của request để chứng minh retrieval và injection thực sự xảy ra, phân biệt với việc dữ liệu chỉ nằm tĩnh trong file trên đĩa cứng.
3. **Phần mở rộng giao diện Web:**
   - Hỗ trợ trò chuyện đa phiên (multiple chats theo project), tự động trích xuất quyết định từ câu thoại thông thường, tab hiển thị memory active và lịch sử superseded, cửa sổ xem trace ngữ cảnh đã nạp, và xuất bằng chứng JSON. Đây là công cụ hỗ trợ trải nghiệm và kiểm chứng, không thay thế hai mục tiêu cốt lõi của đề gốc.

2.2. Bảng đối chiếu tiêu chí và mức độ hoàn thành

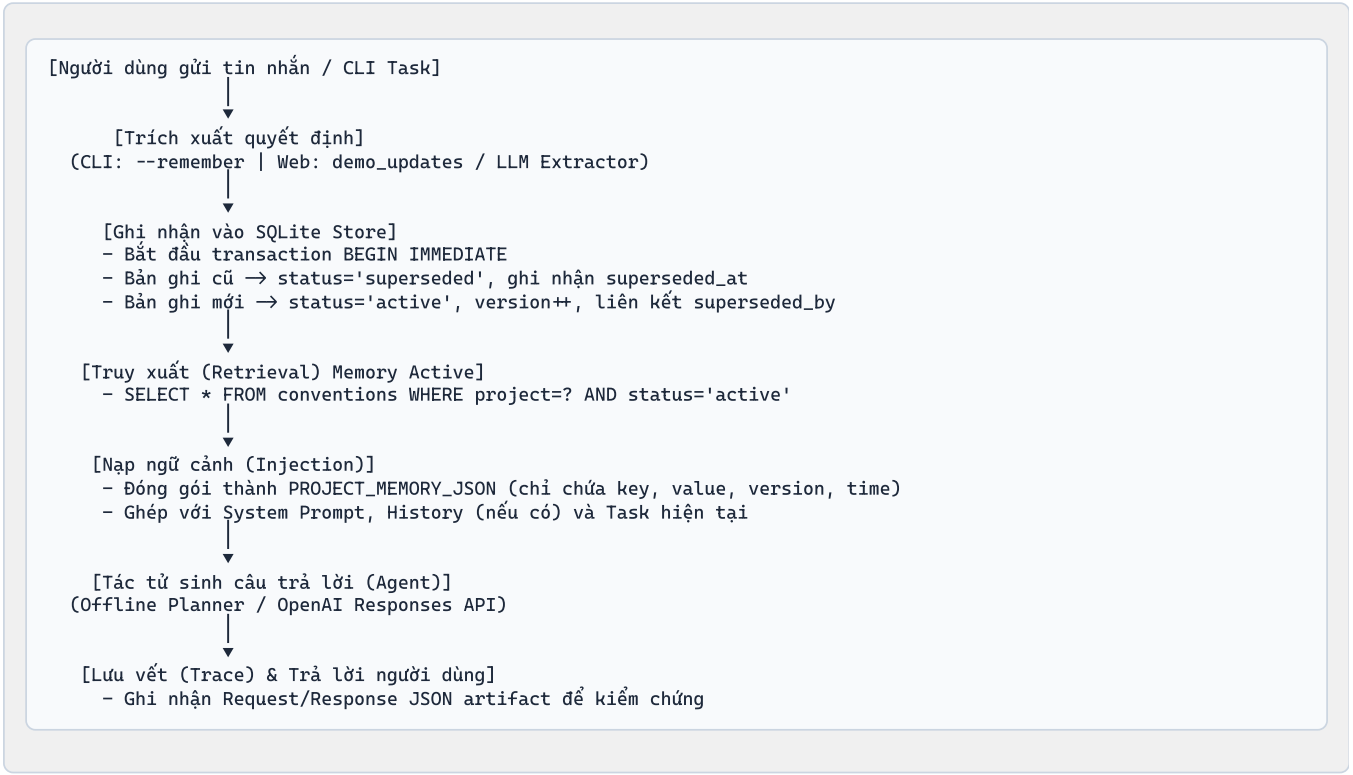
Yêu cầu	Nhóm mục tiêu	Cách triển khai trong mã nguồn	Bằng chứng thực tế	Đánh giá & Giới hạn
Chứng minh 1: Session mới dùng đúng quy ước	Yêu cầu cốt lõi 1	CLI: Mở tiến trình Python mới, truy xuất SQLite và nạp `PROJECT_MEMORY_JSON` vào request. Web: Chat mới có history rỗng nhưng nạp memory active của project.	CLI: `s2-cold-start` (PID 31124) nhớ đủ 4 quy ước v1. Web: Turn 1 của Chat mới (`37ded979 ...`) nhớ đủ stack TypeScript, PostgreSQL, JWT.	**ĐẠT** trên tầng harness và demo logic. Với LLM thật: đã có adapter và hợp đồng mock, chưa có artifact chạy thực với OpenAI API.
Chứng minh 2: Session sau dùng quy ước mới	Yêu cầu cốt lõi 2	Khi quy ước đổi, bản cũ chuyển sang `superseded`, bản mới mang `version 2` và `status='active'`. Request ở session sau chỉ nạp bản active.	CLI: `s4-cold-start` (PID 8156) và `s5-cold-start` (PID 13240) dùng session cookie; request không chứa chuỗi "jwt". Web: Chat `73464a63 ...` chỉ nạp session cookie.	**ĐẠT** trên tầng harness và demo logic. Request hoàn toàn sạch token cũ. Với LLM thật: cần kiểm chứng thủ công ngữ nghĩa câu trả lời tự do.
Xử lý xung đột và lưu trữ nguồn gốc	Quản lý phiên bản & Xung đột	Bảng `conventions` có metadata `version`, `status`, `source`, `created_session`, `created_at`, `superseded_at`, `superseded_by`. Dùng transaction `BEGIN IMMEDIATE`.	`memory-history.json` ghi nhận JWT (v1, superseded) trỏ `superseded_by=5` sang session cookie (v2, active).	**ĐẠT**. Đảm bảo tính toán vẹn lịch sử (provenance audit trail), không xóa trắng hay để 2 quy ước mâu thuẫn cùng active.
Chứng minh Context Injection	Kiểm chứng nạp ngữ cảnh	Khối `PROJECT_MEMORY_JSON` được in ra console stdout khi chạy CLI và lưu trong file JSON của từng session; Web có nút "Ngữ cảnh đã nạp" xem trace.	CLI: Mục `=== MEMORY INJECTED INTO CONTEXT ===` trong từng log `s1` – `s5`. Web: Modal trace và ảnh chụp `03-injected-context.png`.	**ĐẠT**. Chứng minh rõ ràng dữ liệu được harness inject vào context thực tế của request.
Cô lập dữ liệu theo dự án (Project Isolation)	Phân vùng dữ liệu	Bảng `conventions` và bảng `chats` đều có trường `project`. Truy vấn luôn lọc `WHERE project=?`.	Unit test `test_project_isolation` đạt; Browser QA check `separate_project_has_empty_memory` đạt.	**ĐẠT**. Dự án mới có memory hoàn toàn rỗng, không bị lẫn lộn quy ước giữa các dự án khác nhau.

3. KIẾN TRÚC VÀ THIẾT KẾ MEMORY

3.1. Phân định bốn thành phần trạng thái trong kiến trúc Agent

Hệ thống phân định rạch ròi 4 khái niệm nền tảng theo đúng mô hình kiến trúc của môn học:
- **State (Đang làm đến đâu?):** Thể hiện qua trạng thái của từng lượt xử lý hội thoại (`pending` , `completed` , `failed` trong bảng `turns` , mã nguồn webapp.py:L40-L46).
- **History (Đã có tương tác gì?):** Là chuỗi hội thoại qua lại giữa user và assistant trong **chính cuộc trò chuyện hiện tại** (webapp.py:L141-L145). Chat mới mở ra sẽ có history rỗng.
- **Memory (Điều gì cần nhớ lâu dài?):** Các quyết định và quy ước kỹ thuật cốt lõi của dự án được lưu trong bảng `conventions` của MemoryStore. Không phải mọi dòng chat đều biến thành memory.
- **Context (LLM thấy gì trong lượt gọi này?):** Khối thông tin tổng hợp được gửi đến model gồm: System Instructions + Khối Memory Active (`PROJECT_MEMORY_JSON`) + History của chat hiện tại + Tin nhắn mới (memory_agent.py:L132-L148 và chat_engine.py:L143-L147).

3.2. Sơ đồ chuỗi xử lý (Memory Lifecycle)



3.3. Thiết kế lược đồ cơ sở dữ liệu SQLite và Transaction

Để giải quyết bài toán lưu trữ và xung đột phiên bản, module memory_agent.py tạo bảng `conventions` với cấu trúc:

```sql
CREATE TABLE IF NOT EXISTS conventions (
    id INTEGER PRIMARY KEY,
    project TEXT NOT NULL,
    key TEXT NOT NULL,
    value TEXT NOT NULL,
    version INTEGER NOT NULL CHECK(version > 0),
    status TEXT NOT NULL CHECK(status IN ('active', 'superseded')),
    source TEXT NOT NULL,
    created_session TEXT NOT NULL,
    created_at TEXT NOT NULL,
    superseded_at TEXT,
    superseded_by INTEGER REFERENCES conventions(id),
    UNIQUE(project, key, version)
);
CREATE UNIQUE INDEX IF NOT EXISTS one_active_convention
ON conventions(project, key) WHERE status = 'active';
```

Các đặc tính thiết kế trọng yếu:

1. **Chỉ mục bộ phận độc nhất (Partial Unique Index):** Chỉ mục `one_active_convention` áp dụng ràng buộc `WHERE status = 'active'`, đảm bảo tuyệt đối ở mức database rằng tại mọi thời điểm, một cặp `(project, key)` chỉ có tối đa duy nhất một phiên bản có hiệu lực.
2. **Transaction nguyên tử cấp cao (`BEGIN IMMEDIATE`):** Trong phương thức `remember_many` (memory_agent.py:L89-L118), transaction khóa ghi ngay từ đầu. Việc đánh dấu bản cũ bị thay thế và chèn bản mới diễn ra trong cùng một đơn vị công việc. Nếu bất kỳ lỗi nào xảy ra (ví dụ: lỗi khóa ngoại, trigger vi phạm), toàn bộ thay đổi sẽ được rollback, ngăn chặn tình trạng bản cũ bị hủy mà bản mới không được ghi.
3. **Tính lũy kế và Idempotency:** Nếu người dùng nhắc lại một quyết định có cùng `value`, hệ thống ghi nhận hành động `unchanged` và không tăng version. Ngược lại, nếu người dùng quay lại một giá trị trong quá khứ (ví dụ: chuyển lại từ cookie sang JWT), hệ thống tạo ra `version 3` mới với nguồn gốc và thời điểm hiện tại chứ không sửa đổi lịch sử của `version 1`.
4. **Cô lập ngữ cảnh khi Injection:** Hàm `build_request` (memory_agent.py:L132-L148) chỉ lọc lấy 4 trường: `key`, `value`, `version`, `created_at` để đưa vào `PROJECT_MEMORY_JSON`. Tuyệt đối không inject toàn bộ lịch sử `superseded`, không đưa chuỗi `source` (vì câu nguồn có thể nhắc lại từ khóa cũ gây nhiễu cho model), và không đưa database handle vào context.

---

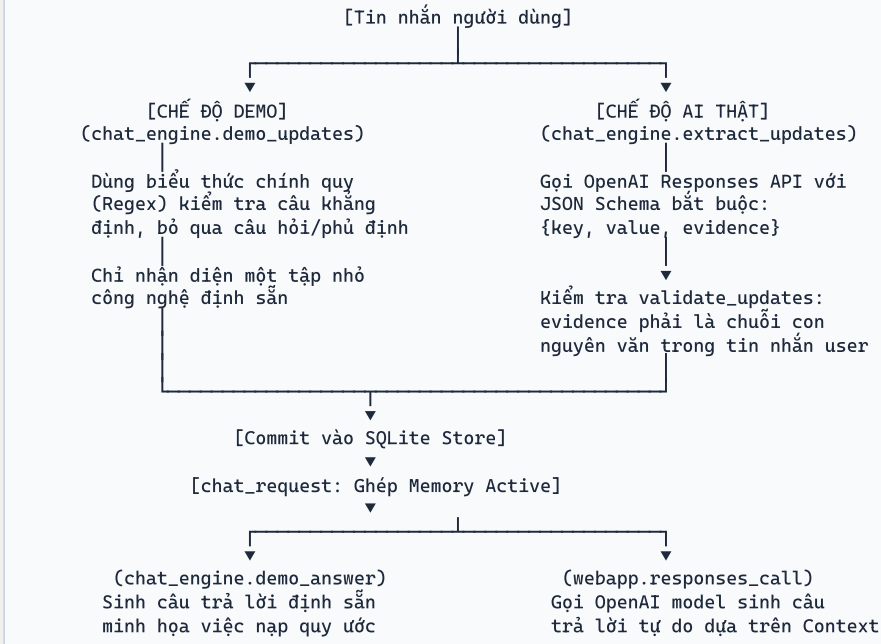
4. LUỒNG HOẠT ĐỘNG TRONG CHATBOT WEB

Ứng dụng web cục bộ (webapp.py) mở rộng mô hình CLI thành giao diện chatbot hiện đại nhưng vẫn tuân thủ nghiêm ngặt nguyên lý stateless/stateful.

4.1. Bốn tình huống tương tác người dùng

1. **Tin nhắn thông thường:** Người dùng gửi câu hỏi kỹ thuật hoặc trao đổi bình thường. Bộ trích xuất không tìm thấy quyết định quy ước nào (`updates = []`). Hệ thống chỉ truy xuất memory active của project hiện tại, kết hợp với lịch sử chat để trả lời.
2. **Tin nhắn chốt quyết định mới:** Người dùng khẳng định quy ước mới (ví dụ: "*Dự án dùng TypeScript, PostgreSQL và JWT*"). Hệ thống trích xuất 3 quyết định, thực hiện `remember_many` lưu vào database, sau đó truy xuất lại memory active mới nạp vào context để trả lời xác nhận.
3. **Tin nhắn thay đổi quy ước:** Người dùng yêu cầu "*Chuyển authentication từ JWT sang session cookie*". Hệ thống cập nhật JWT thành superseded, kích hoạt session cookie v2. Ngay trong câu trả lời của lượt chat đó và các lượt tiếp theo, context chỉ nạp quy ước session cookie mới.
4. **Mở cuộc trò chuyện mới (New Chat):** Tạo một bản ghi mới trong bảng `chats`. Lịch sử hội thoại của chat này hoàn toàn trống (webapp.py:L141-L145). Khi người dùng gửi tin nhắn đầu tiên, harness tự động truy xuất các quy ước active của project từ SQLite để inject vào request. Chat mới không thừa hưởng bất kỳ câu chat nào của chat trước nhưng nắm đầy đủ quy ước của dự án.

4.2. Phân biệt hai chế độ: Demo và AI thật



4.3. Ranh giới giao dịch và an toàn thông tin

- **Không phải Distributed Transaction bao trùm mạng:** Hàm `send` trong webapp.py:L137-L161 tách biệt rõ hai bước. Quyết định quy ước được trích xuất và commit thành công vào SQLite *trước* khi tiến hành gọi mạng sinh câu trả lời. Nếu lượt gọi API bị timeout hoặc lỗi mạng 502, quy ước đã commit vẫn được bảo toàn trong memory, còn lượt chat được lưu trạng thái `failed` cùng thông báo lỗi để người dùng có thể nhấn nút "Thử lại tin nhắn".
- **Bảo mật API Key:** API Key người dùng cấu hình qua giao diện web chỉ tồn tại trong bộ nhớ RAM của tiến trình server (webapp.py:L29). Khóa không bao giờ được ghi xuống đĩa, không lưu vào SQLite, không đưa vào file export và không phản hồi về trình duyệt (webapp.py:L61-L63).
- **Lưu transcript khác với sử dụng memory:** Việc bảng `turns` lưu transcript người dùng chỉ phục vụ hiển thị lịch sử hội thoại trên UI; bằng chứng agent sử dụng memory nằm ở khối `PROJECT_MEMORY_JSON` trong trường `request_json` được lưu vết.

---

5. HAI CHỨNG MINH BẮT BUỘC BẰNG DỮ LIỆU THỰC TẾ

Toàn bộ số liệu và bằng chứng dưới đây được đối chiếu trực tiếp từ lần chạy demo được chỉ định bởi output/LATEST_DEMO.txt: thư mục `output/demo/20261021T023312Z-1fc47e02/`. Thời điểm chạy được ghi nhận lúc **02:33:12Z ngày 21/10/2026** (tức 09:33:12 giờ Việt Nam, UTC+7).

5.1. Bảng đối chiếu 5 tiến trình độc lập (S1–S5)

Bước	Nhãn kịch bản	OS PID	Session UUID	Nội dung Task / Hành động	Memory Active được Inject	Kết quả quan sát từ Agent
S1	`s1-establish`	5356	`e8ca0134918b42b4a210629b0fd6d767`	Chốt 4 quy ước: TypeScript, PostgreSQL, JWT, REST `/api/v1; snake_case JSON`.	4 quy ước v1 vừa tạo.	Ghi nhận 4 quy ước thành công vào SQLite.
S2	`s2-cold-start`	31124	`27365c7e2bf04e74b67bea817f0ac35c`	"Đề xuất kế hoạch triển khai endpoint lấy hồ sơ người dùng."	Đủ 4 quy ước v1 từ S1 (không truyền transcript).	Agent lập kế hoạch dùng TypeScript, PostgreSQL, REST `/api/v1`, bearer token JWT.
S3	`s3-change`	3292	`d532507022b9449e9dbe6f46862c25fe`	Cập nhật `authentication = session cookie`.	3 quy ước cũ v1 + `authentication` session cookie v2.	JWT v1 chuyển thành `superseded`, cookie v2 thành `active`.
S4	`s4-cold-start`	8156	`20b624dd3b3d41129ad95c48c8154eda`	"Đề xuất kế hoạch triển khai endpoint đăng xuất."	4 quy ước active (chứa session cookie v2; không có JWT).	Agent dùng session cookie, tra session phía server.
S5	`s5-cold-start`	13240	`87fd5f01b12d40b6b623c66b1ec2f442`	"Đề xuất kế hoạch triển khai middleware kiểm tra đăng nhập."	4 quy ước active (chứa session cookie v2; không có JWT).	Agent tiếp tục dùng session cookie v2 một cách nhất quán.

Cả 5 session đều được thực thi bằng lệnh `subprocess.run` riêng biệt trong demo.py:L43-L50, tương ứng 5 PID khác nhau (`5356`, `31124`, `3292`, `8156`, `13240`), chứng minh đây là các tiến trình độc lập mở lại SQLite từ đĩa cứng mà không chia sẻ RAM.

5.2. Phân tích chi tiết Chứng minh 1: Session mới giữ đúng quy ước

- **File bằng chứng:** output/demo/20261021T023312Z-1fc47e02/s2-cold-start/27365c7e2bf04e74b67bea817f0ac35c.json và log s2-cold-start.log.
- **Đầu vào của User trong Session 2:**
  `json { "role": "user", "content": "Đề xuất kế hoạch triển khai endpoint lấy hồ sơ người dùng." }`
  (Lưu ý: Trong câu hỏi của user hoàn toàn không chứa các từ khóa như TypeScript, PostgreSQL, JWT hay API style).
- **Ngữ cảnh Memory được nạp vào Request:**
  `json "content": "PROJECT_MEMORY_JSON\n{\n \"project\": \"course-project\",\n \"active_conventions\": [\n {\n \"key\": \"api_style\",\n \"value\": \"REST /api/v1; snake_case JSON\",\n \"version\": 1,\n \"created_at\": \"2026-10-21T02:33:13.026738+00:00\"\n },\n {\n \"key\": \"authentication\",\n \"value\": \"JWT\",\n \"version\": 1,\n \"created_at\": \"2026-10-21T02:33:13.026724+00:00\"\n },\n {\n \"key\": \"database\",\n \"value\": \"PostgreSQL\",\n \"version\": 1,\n \"created_at\": \"2026-10-21T02:33:13.026698+00:00\"\n },\n {\n \"key\": \"language\",\n \"value\": \"TypeScript\",\n \"version\": 1,\n \"created_at\": \"2026-10-21T02:33:13.025704+00:00\"\n }\n]\n}"`
- **Kết quả phản hồi của Agent:** Khối `response.conventions_used` tự động bắt đủ 4 quy ước; kế hoạch đề xuất ghi rõ: *"Authentication: JWT; middleware xác minh bearer token của request"*, *"Ngôn ngữ triển khai: TypeScript"*, *"Cơ sở dữ liệu: PostgreSQL"*.
- **Kết luận:** Chứng minh 1 đạt yêu cầu: Session mới hoàn toàn tự động kế thừa đầy đủ các quy ước kỹ thuật đã lưu trên đĩa mà không cần người dùng nhắc lại.

5.3. Phân tích chi tiết Chứng minh 2: Quy ước đổi thì các session sau dùng quy ước mới

- **File bằng chứng:** output/demo/20261021T023312Z-1fc47e02/s4-cold-start/20b624dd3b3d41129ad95c48c8154eda.json và memory-history.json.
- **Truy vết thay đổi trong Cơ sở dữ liệu (Database Provenance):**
  Trích xuất từ `memory-history.json` cho key `authentication`:
  `json [{ "id": 3, "key": "authentication", "value": "JWT", "version": 1, "status": "superseded", "source": "Demo user S1: thống nhất stack và API", "created_session": "e8ca0134918b42b4a210629b0fd6d767", "created_at": "2026-10-21T02:33:13.026724+00:00", "superseded_at": "2026-10-21T02:33:13.667874+00:00", "superseded_by": 5 }, { "id": 5, "key": "authentication", "value": "session cookie", "version": 2, "status": "active", "source": "Demo user S3: chuyển từ JWT sang session cookie", "created_session": "d532507022b9449e9dbe6f46862c25fe", "created_at": "2026-10-21T02:33:13.667874+00:00", "superseded_at": null, "superseded_by": null }]`
  Quan sát cho thấy: Bản ghi JWT v1 không bị xóa mất mà được cập nhật `status='superseded'`, lưu chính xác thời điểm thay thế và trỏ con trỏ kế vị `superseded_by = 5`. Bản ghi mới `session cookie` có ID 5, mang `version 2`, `status='active'`.
- **Kiểm chứng tại Session 4 (Cold start):**
  User yêu cầu *"Đề xuất kế hoạch triển khai endpoint đăng xuất."* Khối `PROJECT_MEMORY_JSON` được nạp vào S4 chỉ chứa:
  `json { "key": "authentication", "value": "session cookie", "version": 2, "created_at": "2026-10-21T02:33:13.667874+00:00" }`
  Toàn bộ cấu trúc request của S4 và S5 không hề xuất hiện từ khóa "jwt" (`check s4_no_stale_jwt_in_request = true`). Phản hồi của agent chuyển hẳn sang: *"Authentication: session cookie; middleware tra session phía server từ cookie"*.
- **Đối chiếu trên Web QA:** File output/web-qa/browser-export.json chứng minh luồng tương tự trên giao diện đồ họa: Chat mới `73464a63 ...` có request sạch hoàn toàn không chứa JWT, agent giải thích cơ chế session cookie, HttpOnly và CSRF.
- **Lưu ý ranh giới:** Việc khẳng định "không có JWT trong request" áp dụng đối với các request mới được kiểm tra (ở các session/chat mới). Trong các cuộc trò chuyện cũ (như chat nơi người dùng gõ câu lệnh đổi), transcript quá khứ vẫn có thể lưu từ khóa JWT.

---

6. KIỂM THỬ VÀ KẾT QUẢ

Dự án có hệ thống kiểm thử đa tầng được tách bạch rõ ràng theo từng công cụ và mục đích, không gộp lẫn lộn số lượng bài test giữa các môi trường khác nhau:

6.1. Tổng hợp các nhóm kiểm thử

Bộ kiểm thử	Công cụ thực hiện	Quy mô	Kết quả	Nguồn file log / Bằng chứng	Bản chất kiểm chứng
CLI Demo Assertions	Python subprocess (`demo.py`)	15 kiểm tra	**15/15 ĐẠT**	`output/demo/20261021T023312Z-1fc47e02/verification.json`	Kiểm tra 5 tiến trình độc lập, injection sạch, không rò rỉ history, metadata đúng. Provider: `offline`.
Unit Test Memory & CLI	Python `unittest` (`test_memory_agent.py`)	16 tests	**16/16 ĐẠT**	`output/tests.log` (Thời gian: 1.876s)	Kiểm tra transactional rollback, schema index, versioning, isolation, mock API contract.
Unit Test Web App	Python `unittest` (`test_webapp.py`)	12 tests	**12/12 ĐẠT**	Gộp trong `output/web-tests.log` (Tổng 28 tests trong 3.817s)	Kiểm tra HTTP routes, idempotency, natural extraction, bảo vệ API key ngoài DB, restart server.
Browser E2E QA	Playwright & Edge headless (`browser-check.cjs`)	13 kiểm tra	**13/13 ĐẠT**	`output/web-qa/verification.json`, 6 ảnh chụp màn hình	Kiểm tra toàn diện UI: gửi tin nhắn, mở chat mới nhớ memory, xem trace context, export JSON, mobile viewport.

(Ghi nhận riêng: File log `output/demo-run.log` thuộc về lần chạy thử nghiệm trước đó `20261021T021735Z-83c61c69`. Báo cáo này căn cứ chính thức trên lần chạy latest `20261021T023312Z-1fc47e02` để đảm bảo tính nhất quán của số hiệu PID và Session ID).

6.2. Phân tích các ca kiểm thử trọng điểm (Key Test Cases)

1. **Kiểm tra Rollback khi lỗi ghi** (test_memory_agent.py:L90-L101):
   Ca kiểm thử `test_failed_insert_rolls_back_retirement_and_other_decisions` tạo một trigger SQLite nhân tạo gây lỗi khi chèn bản ghi mới. Kết quả xác nhận transaction `BEGIN IMMEDIATE` phục hồi nguyên trạng: bản ghi cũ không bị chuyển sang `superseded` oan uổng.
2. **Kiểm tra ngăn chặn 2 bản ghi Active** (test_memory_agent.py:L102-L110):
   Ca kiểm thử `test_sqlite_prevents_two_active_versions` cố tình chèn trực tiếp một bản ghi active thứ hai vào SQLite. Ràng buộc `UNIQUE INDEX one_active_convention` chặn đứng và ném lỗi `IntegrityError`.
3. **Kiểm tra khôi phục giá trị cũ tạo Version mới** (test_memory_agent.py:L111-L118):
   Ca kiểm thử `test_reverting_to_old_value_creates_new_version` đổi quy ước theo chuỗi JWT -> Cookie -> JWT. Hệ thống sinh ra 3 phiên bản [1, 2, 3] và bản active hiện hành mang version 3, bảo toàn audit trail.
4. **Kiểm tra cô lập Project** (test_memory_agent.py:L82-L89):
   Ca kiểm thử `test_project_isolation` chứng minh việc lưu quy ước ở project 'a' và project 'b' không ảnh hưởng lẫn nhau; truy vấn project 'c' trả về mảng rỗng.
5. **Kiểm tra khởi động lại ứng dụng** (test_webapp.py:L97-L110):
   Ca kiểm thử `test_restart_retains_chats_and_memory_but_not_config_key` dừng HTTP server và tạo lại đối tượng app từ file SQLite trên đĩa. Dữ liệu chat và memory được bảo toàn nguyên vẹn, trong khi API key lưu ở RAM biến mất. *(Lưu ý: Đây là kiểm thử tái tạo đối tượng server trong tiến trình test, không phải khởi động lại hệ điều hành).*
6. **Kiểm tra luồng AI hai giai đoạn** (test_webapp.py:L163-L183):
   Ca kiểm thử `test_ai_uses_two_stages_and_updated_memory_in_final_request` dùng mock để xác nhận harness thực hiện đúng 2 lượt gọi: Lượt 1 trích xuất JSON Schema -> Commit DB -> Lượt 2 gửi request hoàn chỉnh chứa memory đã cập nhật.

---

7. HẠN CHẾ VÀ KẾ HOẠCH KIỂM CHỨNG BỔ SUNG

7.1. Các hạn chế kỹ thuật hiện tại

Báo cáo ghi nhận trung thực các hạn chế dựa trên mã nguồn và dữ liệu thực tế:
1. **Bộ trích xuất Demo dựa trên quy tắc:** Hàm `demo_updates` và `demo_answer` sử dụng regex đơn giản để phục vụ trình diễn; nó không phải là mô hình trí tuệ nhân tạo và không thể hiểu các câu đàm thoại tiếng Việt phức tạp, nhiều tầng nghĩa.
2. **Hạn chế của Extractor dùng AI:** Bước kiểm tra chuỗi con `evidence in text` (chat_engine.py:L92) chỉ bảo đảm mô hình không bịa đặt nguồn trích dẫn, nhưng không thể bảo đảm 100% mô hình hiểu đúng ngữ cảnh phủ định, giả định hoặc suy luận chính xác giá trị quy ước trong mọi trường hợp.
3. **Lịch sử trong Chat cũ:** Trong một cuộc trò chuyện kéo dài qua nhiều lần đổi quy ước, transcript quá khứ vẫn chứa các từ khóa lỗi thời. Mặc dù system prompt đã chỉ thị mô hình ưu tiên khối memory active, các LLM nhỏ vẫn có nguy cơ bị phân tâm bởi lịch sử cũ (context distraction).
4. **Mô hình phục vụ:** Hệ thống hiện là ứng dụng chạy cục bộ trên máy cá nhân (localhost), đơn người dùng, chưa có cơ chế xác thực tài khoản phân quyền hoặc đồng bộ phân tán.
5. **Khoảng trống bằng chứng LLM thật:** Trong bộ dữ liệu artifact bàn giao, toàn bộ kịch bản đều chạy dưới chế độ `offline` hoặc `demo`, và các bài test API đều sử dụng `mock`. Hiện chưa có artifact ghi nhận kết quả gọi LLM thương mại thật (OpenAI) cho cả hai tình huống của Challenge A.

7.2. Kế hoạch kiểm chứng bổ sung với LLM thật (Verification Plan)

Để kiểm chứng năng lực tuân thủ thực tế của một mô hình ngôn ngữ lớn thương mại, quy trình sau được đề xuất để thực hiện trong tương lai:
1. **Chuẩn bị môi trường:** Khởi tạo một dự án độc lập trên Web App (ví dụ: `LLM-Verification-Project`). Thiết lập API Key và Model trong menu *Cài đặt AI* (ví dụ: `gpt-4o` hoặc `gpt-5-mini`).
2. **Kịch bản thực thi:**
- *Chat 1:* Nhập câu thoại tự nhiên chốt stack: "*Dự án này chúng ta thống nhất dùng TypeScript, PostgreSQL và xác thực bằng JWT.*" Kiểm tra xem model có trích xuất đúng 3 quy ước vào bảng bên phải hay không.

- *Chat 2 (Kiểm chứng 1):* Mở cuộc trò chuyện mới, hỏi: "*Hãy hướng dẫn tôi tạo endpoint đăng nhập cho dự án.*" Đánh giá câu trả lời của model có tự động viết mã TypeScript và sinh token JWT mà không hỏi lại người dùng hay không.
- *Chat 2 tiếp tục:* Nhập tin nhắn đổi quy ước: "*Chúng ta bỏ JWT, chuyển hoàn toàn sang session cookie nhé.*" Xác nhận bảng memory chuyển JWT thành `superseded` và session cookie thành `active`.
- *Chat 3 (Kiểm chứng 2):* Mở cuộc trò chuyện mới khác, hỏi: "*Viết middleware xác thực cho API.*" Đánh giá câu trả lời xem model có sử dụng cookie parser và session store phía server hay không; kiểm tra xem model có tuyệt đối không nhắc đến bearer token JWT hay không.

3. **Thu thập bằng chứng:** Nhấn nút "*Xuất bằng chứng Challenge A*" trên giao diện để lưu file JSON chứa toàn bộ request, response, token usage và snapshot database để đối chiếu.

---

8. KẾT LUẬN

Qua quá trình nghiên cứu tài liệu môn học, phân tích mã nguồn và kiểm chứng bằng chứng thực tế, báo cáo rút ra các kết luận sau:

1. **Về mặt kiến trúc phần mềm và Agent Harness:**
   Đồ án đã giải quyết hoàn chỉnh, triệt để hai yêu cầu chứng minh của **Challenge A**:
   - Cơ chế lưu trữ SQLite với chỉ mục bộ phận độc nhất (`one_active_convention`) và transaction `BEGIN IMMEDIATE` đảm bảo tính toàn vẹn dữ liệu, chống xung đột phiên bản và ngăn chặn ghi dở dang khi có lỗi.
   - Cơ chế giải quyết xung đột (Conflict Resolution) tuân thủ đúng nguyên lý thiết kế: không append hai quy ước mâu thuẫn cùng lúc, không xóa sạch lịch sử, mà đánh dấu `superseded` kèm đầy đủ thông tin nguồn gốc, thời điểm và liên kết phiên bản.
   - Cơ chế nạp ngữ cảnh (Context Injection) được chứng minh rõ ràng: request gửi đi chỉ chứa các quy ước đang có hiệu lực (`active`), hoàn toàn loại bỏ các quy ước đã lỗi thời, đảm bảo agent ở phiên mới luôn hành động đúng theo thỏa thuận kỹ thuật mới nhất.
2. **Về mặt thực thi và kiểm thử:**
   Hệ thống sở hữu bộ bằng chứng thực nghiệm phong phú, nhất quán: 15 assertions trên 5 tiến trình độc lập của CLI Demo đạt tuyệt đối; 28 bài unit/integration test đạt 100%; 13 bước kiểm thử trình duyệt tự động kèm ảnh chụp màn hình xác nhận giao diện hoạt động chính xác, thẩm mỹ và an toàn.
3. **Về mặt mô hình ngôn ngữ lớn:**
   Cần phân định khách quan rằng các kết luận trên chứng minh tính đúng đắn của **lớp điều phối (Harness) và cơ chế bộ nhớ (Memory Persistence)**. Do chưa có artifact chạy thực tế với OpenAI API trong bộ dữ liệu bàn giao, năng lực tuân thủ ngữ nghĩa tự nhiên của mô hình ngôn ngữ trong các ngữ cảnh phức tạp cần được đánh giá bổ sung theo quy trình kiểm chứng đã nêu.

PHỤ LỤC: TÁI HIỆN VÀ TRA CỨU NGUỒN

A.1. Các lệnh thực thi trong môi trường PowerShell

Khởi động ứng dụng Web Chat (mặc định tại cổng 8765):

```
python -X utf8 webapp.py
```

(Hoặc sử dụng script hỗ trợ: `.\start-web.ps1`)

Chạy kịch bản kiểm chứng tự động 5 tiến trình của Challenge A:

```
python -X utf8 demo.py
```

Chạy toàn bộ 28 bài kiểm thử tự động (Unit & Integration tests):

```
python -X utf8 -m unittest discover -s tests -v
```

A.2. Trình tự tái hiện hai chứng minh trên giao diện Web Chat

1. Truy cập `http://127.0.0.1:8765` trên trình duyệt web.
2. Tại khung chat đầu tiên, nhập:
   `text Dự án dùng TypeScript, PostgreSQL và JWT. Chúng ta chọn API REST /api/v1 với JSON snake_case.`
   Quan sát bảng bên phải: 4 thẻ quy ước xuất hiện với nhãn `v1`.
3. Nhấn nút **"+ Cuộc trò chuyện mới"** ở thanh điều hướng bên trái.
4. Trong cuộc trò chuyện mới, nhập:
   `text Hãy lên kế hoạch endpoint hồ sơ theo quy ước dự án.`
   Nhấn nút **"Ngữ cảnh đã nạp"** ở góc dưới bên phải hoặc trên tin nhắn để kiểm tra khối JSON request: xác nhận đủ 4 quy ước v1 được nạp vào context mà không có transcript cũ (*Chứng minh 1*).
5. Quay lại chat trước hoặc gõ ngay trong chat hiện tại:
   `text Chuyển authentication từ JWT sang session cookie.`
   Quan sát bảng bên phải: thẻ `authentication` cập nhật thành `v2 · session cookie`. Chuyển sang tab **"Lịch sử"** để thấy thẻ JWT chuyển màu xám với nhãn `v1 · Đã thay thế`.
6. Nhấn nút **"+ Cuộc trò chuyện mới"** lần nữa, nhập:
   `text Hãy lập kế hoạch middleware đăng nhập theo quy ước dự án.`
   Mở **"Ngữ cảnh đã nạp"**: xác nhận request chỉ chứa `session cookie`, hoàn toàn không có chuỗi `JWT` (*Chứng minh 2*).
7. Nhấn nút **"Xuất bằng chứng Challenge A"** để tải file JSON đối chiếu.

A.3. Bảng ánh xạ tập tin và đường dẫn mã nguồn

Thành phần chức năng	Tập tin mã nguồn / Bằng chứng	Dòng tham chiếu chính
Lớp lưu trữ MemoryStore	memory_agent.py	Schema: L47–L64; `remember_many`: L72–L118; `active`: L120–L124; `history`: L126–L130
Đóng gói Request Context	memory_agent.py	`build_request`: L132–L148
Xử lý hội thoại & Trích xuất	chat_engine.py	`demo_updates`: L51–L79; `validate_updates`: L82–L97; `chat_request`: L143–L147
Ứng dụng Web & API Server	webapp.py	`ChatApp.send`: L117–L161; `Handler.do_GET`: L202–L224; `Handler.do_POST`: L225–L253
Kịch bản Demo 5 tiến trình	demo.py	Kịch bản S1–S5: L34–L60; Các assertions: L65–L90
Kiểm thử tự động	tests/test_memory_agent.py tests/test_webapp.py	16 bài test persistence & CLI contract; 12 bài test web chat & API security
Artifact Demo chính thức	output/demo/20261021T023312Z-1fc47e02/	`verification.json`, `memory-history.json`, log và JSON từ S1 đến S5
Artifact Trình duyệt Web	output/web-qa/	`verification.json`, `browser-export.json`, 6 ảnh chụp màn hình `01` đến `06`